

New IP Architectures & Mobility (TSM_AdvComArc)

Multipath TCP

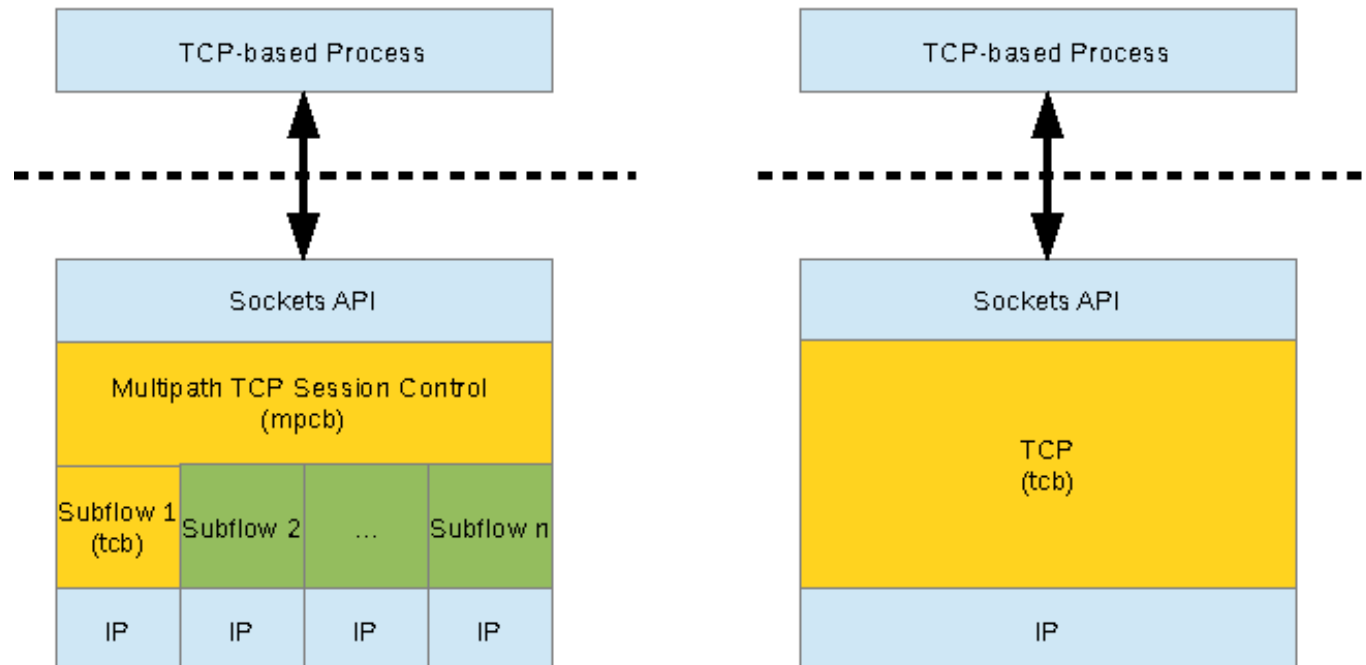
Introduction

TCP's congestion control adapts the window to fit the capacity available in the network.

- Each round-trip time, increase window by one packet.
- If a packet is lost, halve the window.
- TCP reacts to congestion, but does not prevent congestion.
- If an Internet link fails, routing can switch to a different path.
 - Routing doesn't know about congestion.
- Redundancy is the only way to make reliability greater than that of all the parts.
 - Can we use redundancy at TCP level?
 - Can we avoid congestion at TCP level?

Introduction

- Multipath TCP is a major extension to TCP that allows improving the resource usage in the current internet by transmitting data over several TCP subflows.



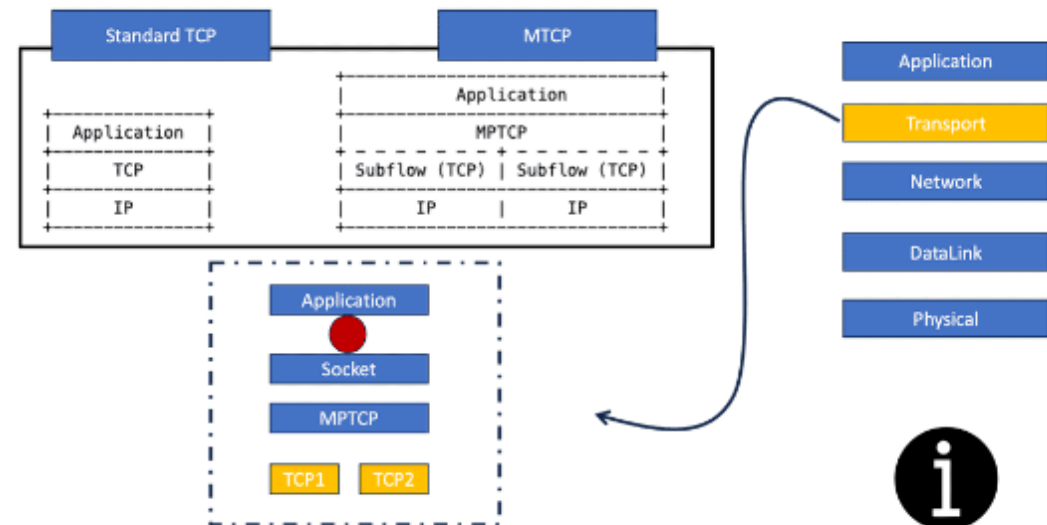
Introduction

- Multipath TCP was designed to be backward compatible with regular TCP. As such, it can support any application.
- MPTCP offers a number of advantages for devices that have more than one network interface available.
- It is possible to transport data simultaneously over more than one interface in order to increase the transfer speed.
- Multipath TCP(MPTCP) is implemented within the kernel and is designed to be backwards compatible with existing TCP socket APIs.

Beyond single-path limits

- **Simultaneous Subflows:** MPTCP enables data transmission across multiple interfaces (Wi-Fi, 4G/5G) on a single connection.
- **Transparent Operation:** Appears as regular TCP to applications while managing complex subflow logistics internally.
- **Inverse Multiplexing:** Increases total throughput to the sum of all available link capacities.

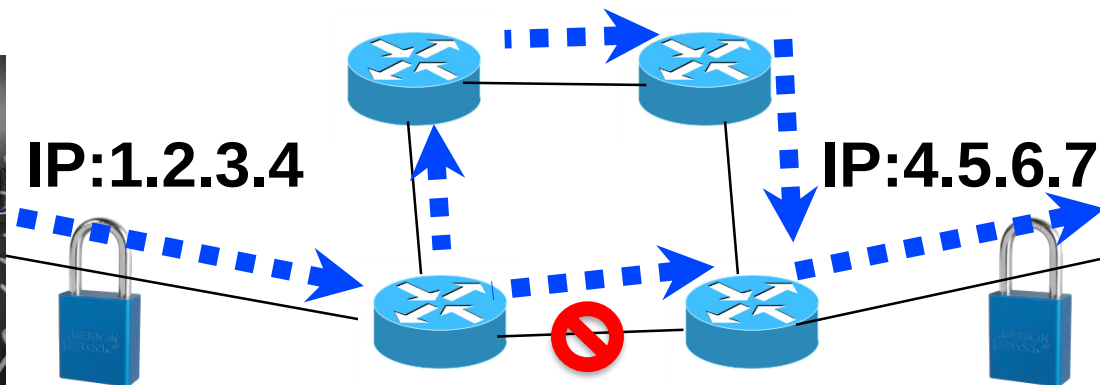
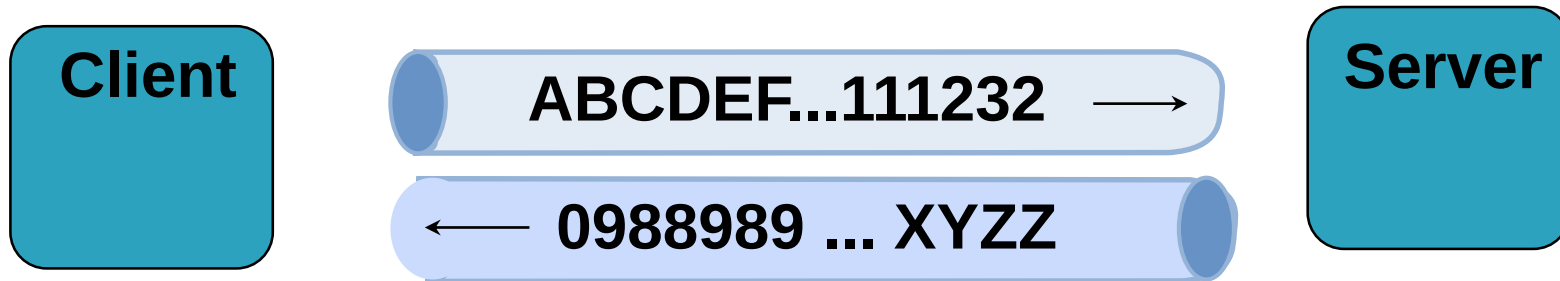
$$BW_{total} = \sum_{i=1}^n bw_i$$



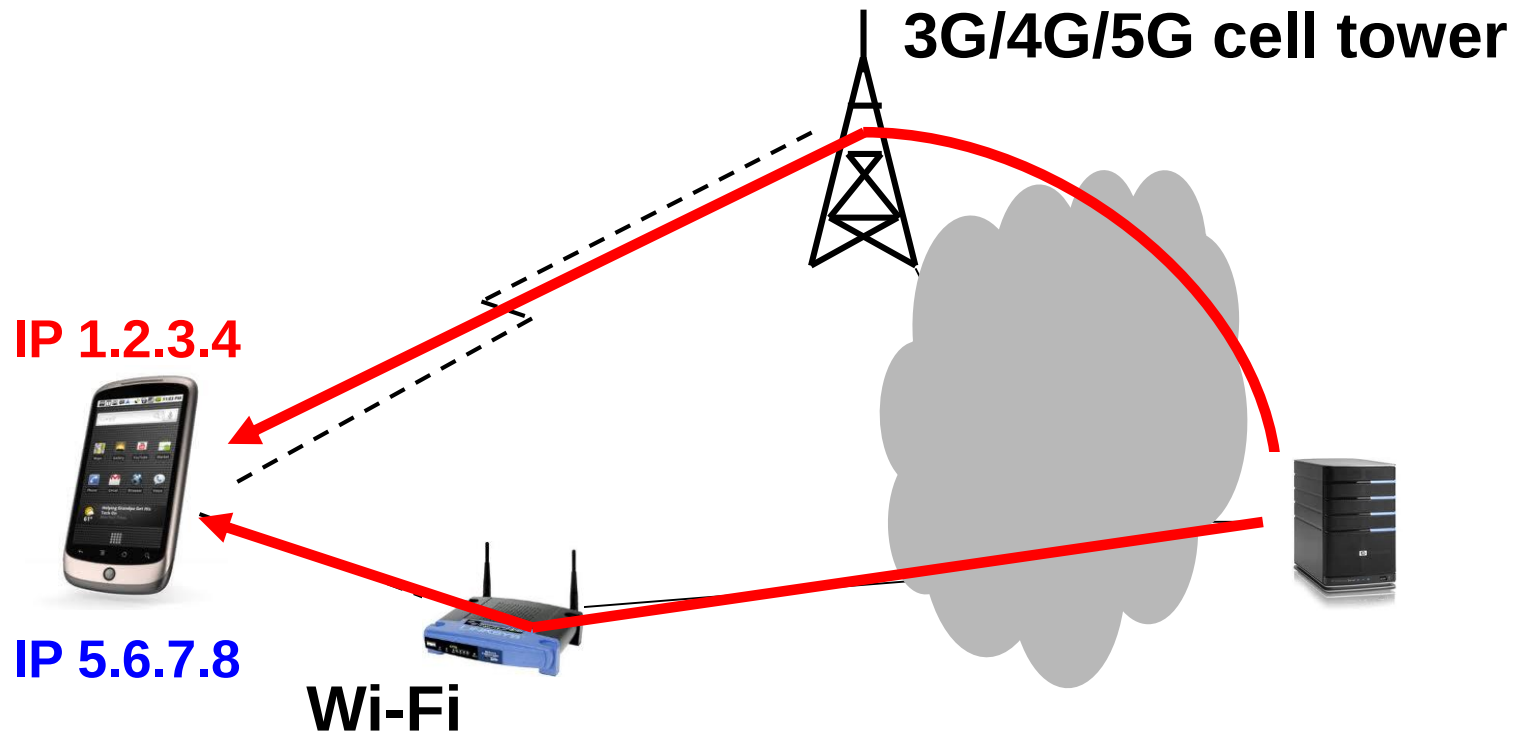
Some updates

- Apple's macOS/iOS was the first major player with official support, Apple's iOS7 supports Multipath TCP for its traffic generated by Siri.
- Linux kernel 5.6 (March 2020) announced MPTCP support as mainline.
- Interesting links:
 - https://en.wikipedia.org/wiki/Multipath_TCP
 - <https://www.rfc-editor.org/rfc/rfc8684>
 - <https://access.redhat.com/MPTCP>

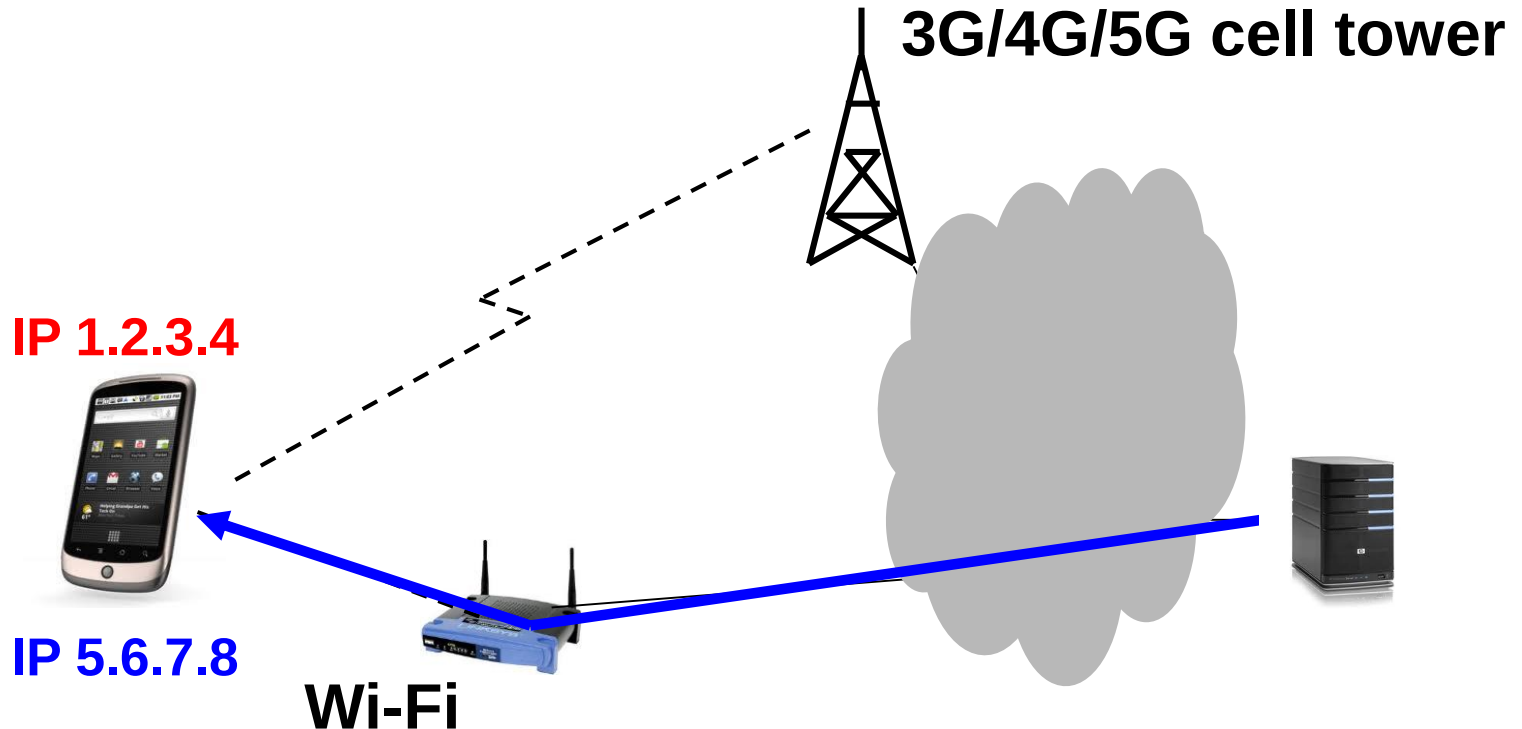
The TCP bytestream model



Connectivity: what technology provides



Connectivity: what technology provides



**When IP addresses change TCP connections
have to be re-established !**

Equal Cost Multipath



ECMP implementation

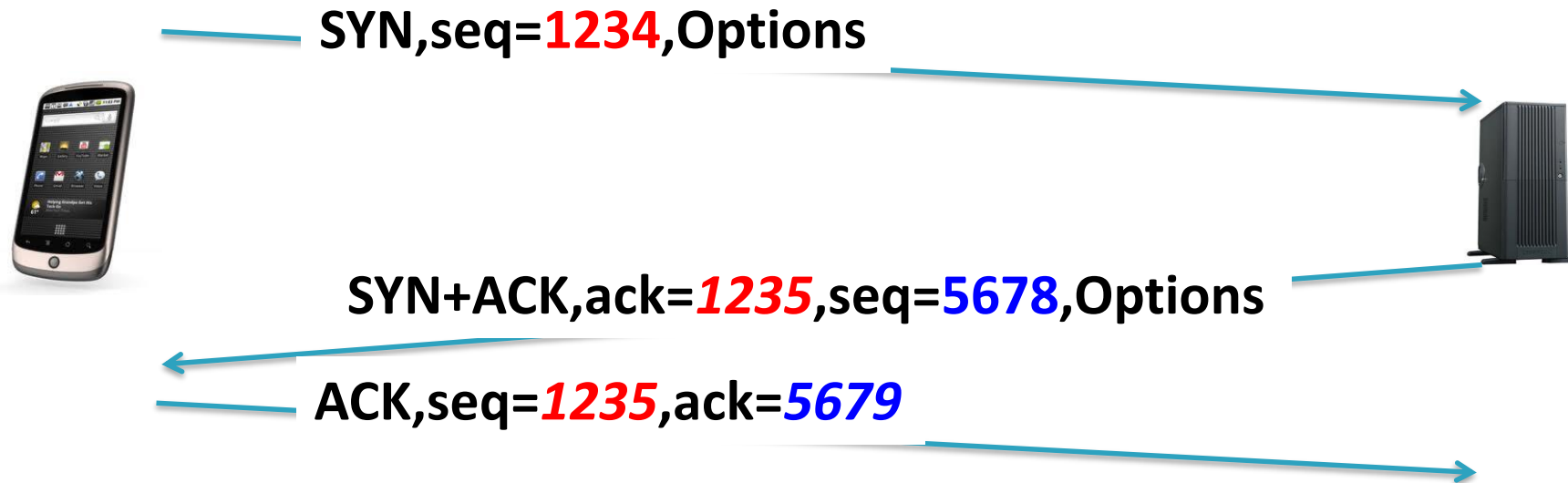
Packet arrival :

$\text{Hash}(\text{IP}_{\text{src}}, \text{IP}_{\text{dst}}, \text{Prot}, \text{Port}_{\text{src}}, \text{Port}_{\text{dst}}) \bmod \# \text{oif}$

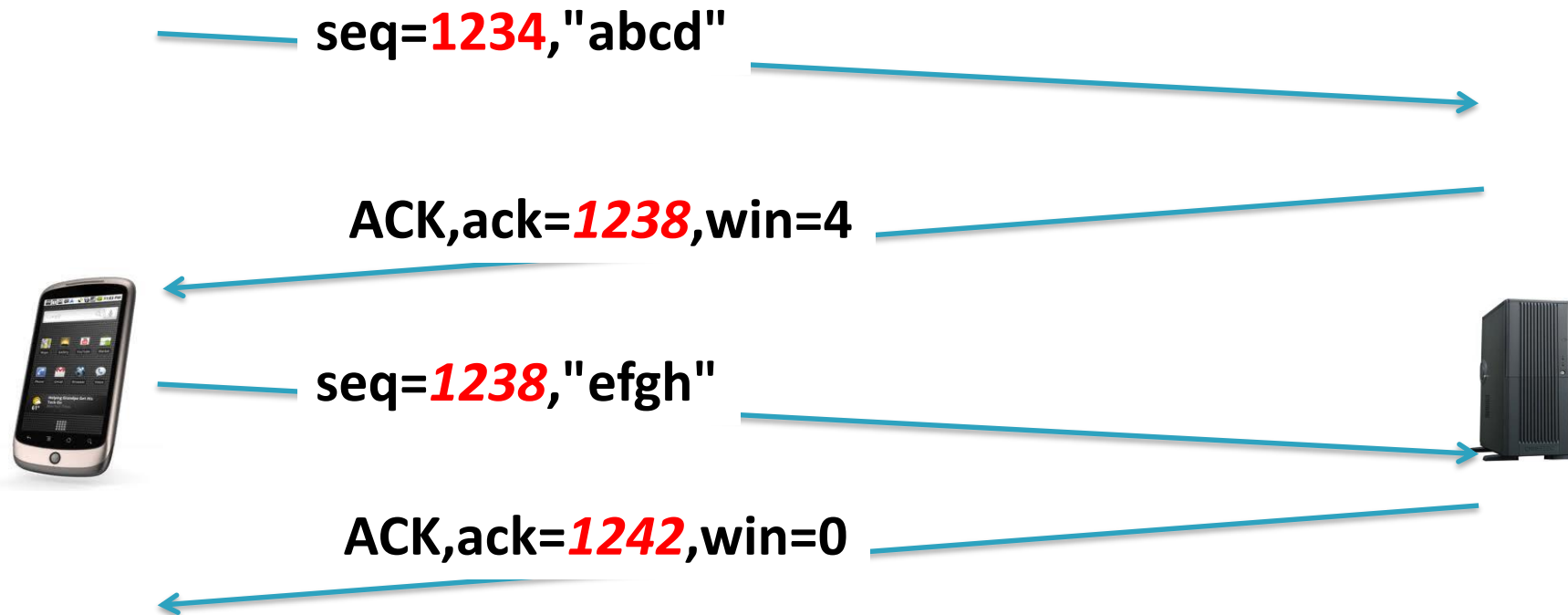
Packets from one TCP connection follow same path
Different connections follow different paths

TCP Connection establishment

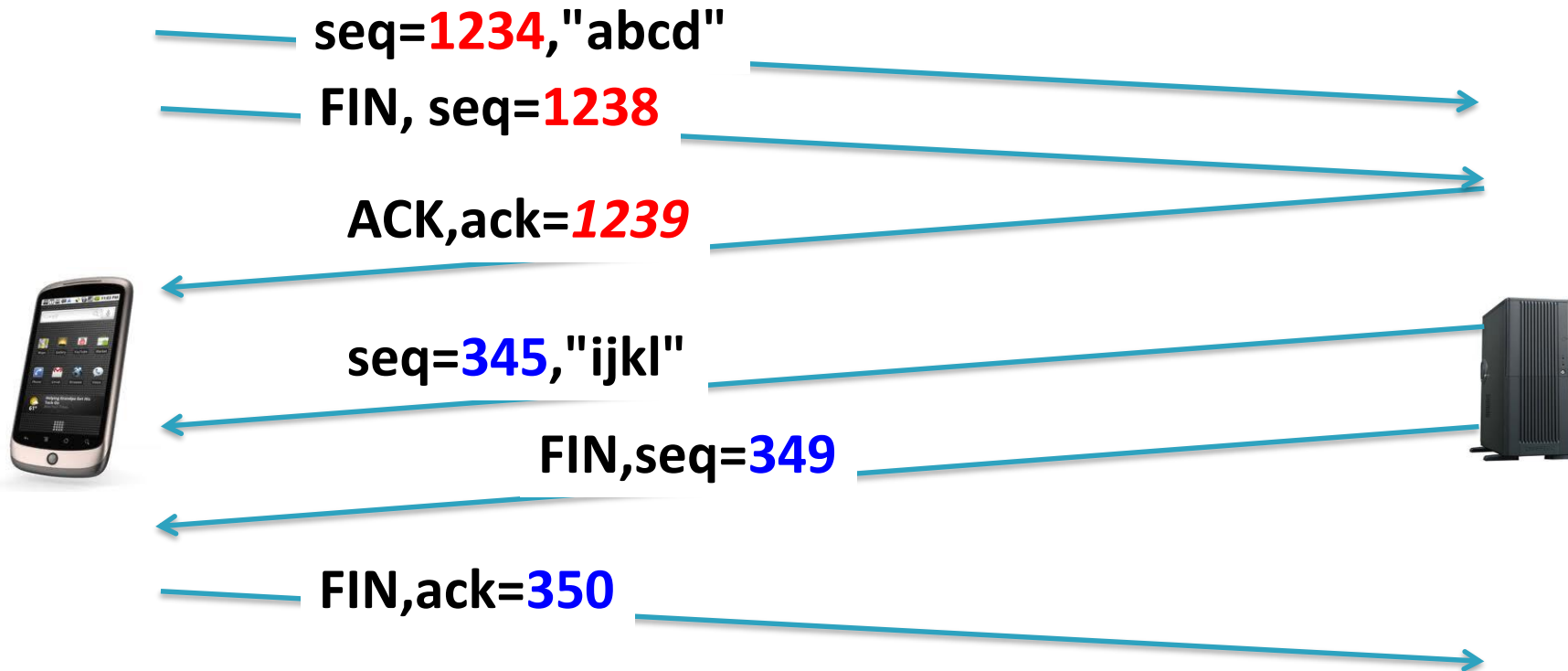
- Three-way handshake



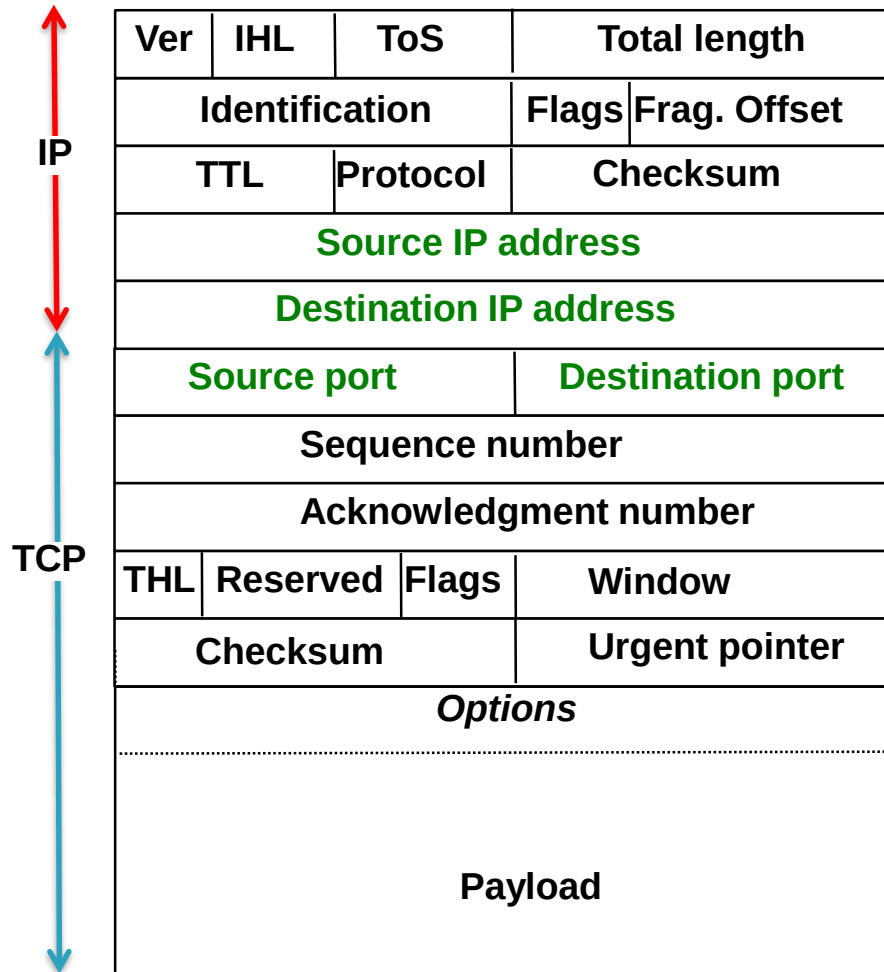
Data transfer



Connection release



Identification of a TCP connection



Four tuple

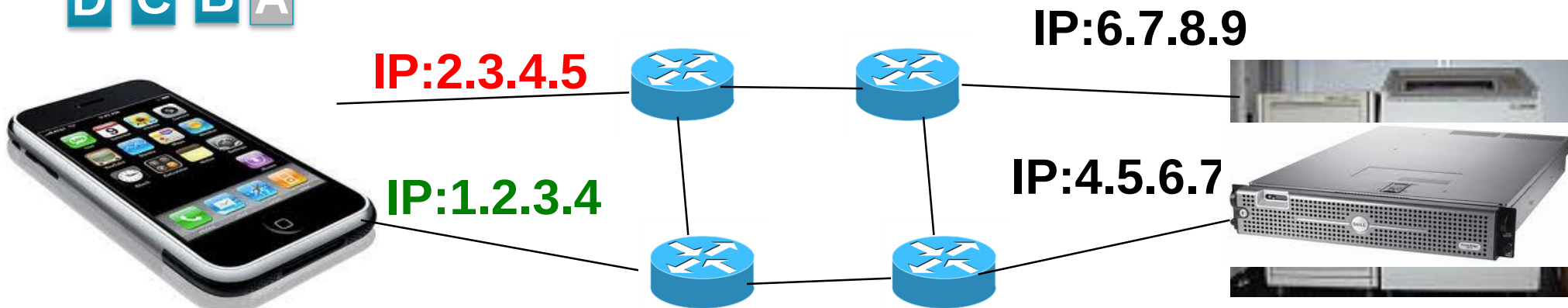
- IP_{source}
- IP_{dest}
- $Port_{source}$
- $Port_{dest}$

All TCP segments contain the four tuple

The new bytestream model



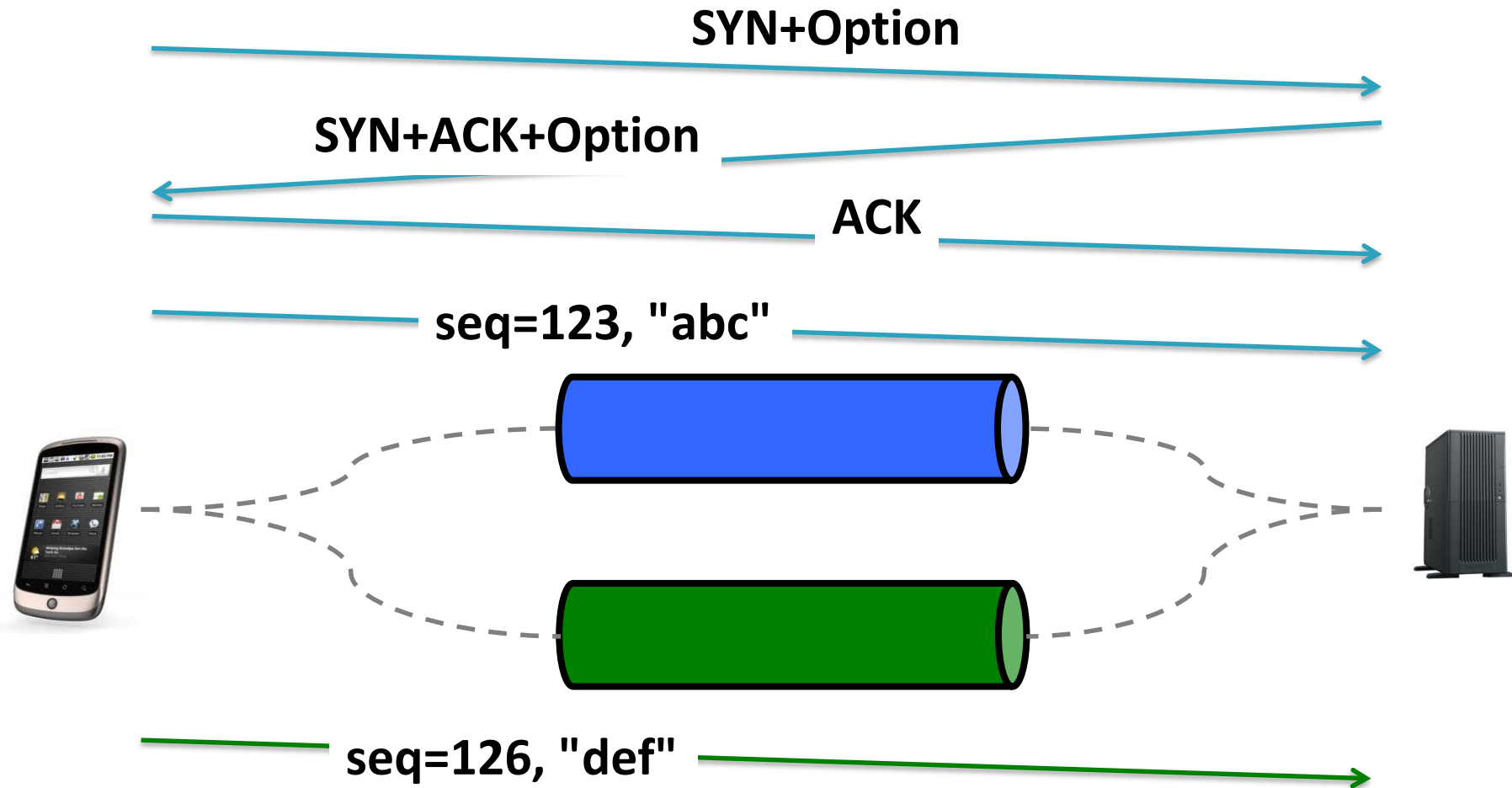
D C B A



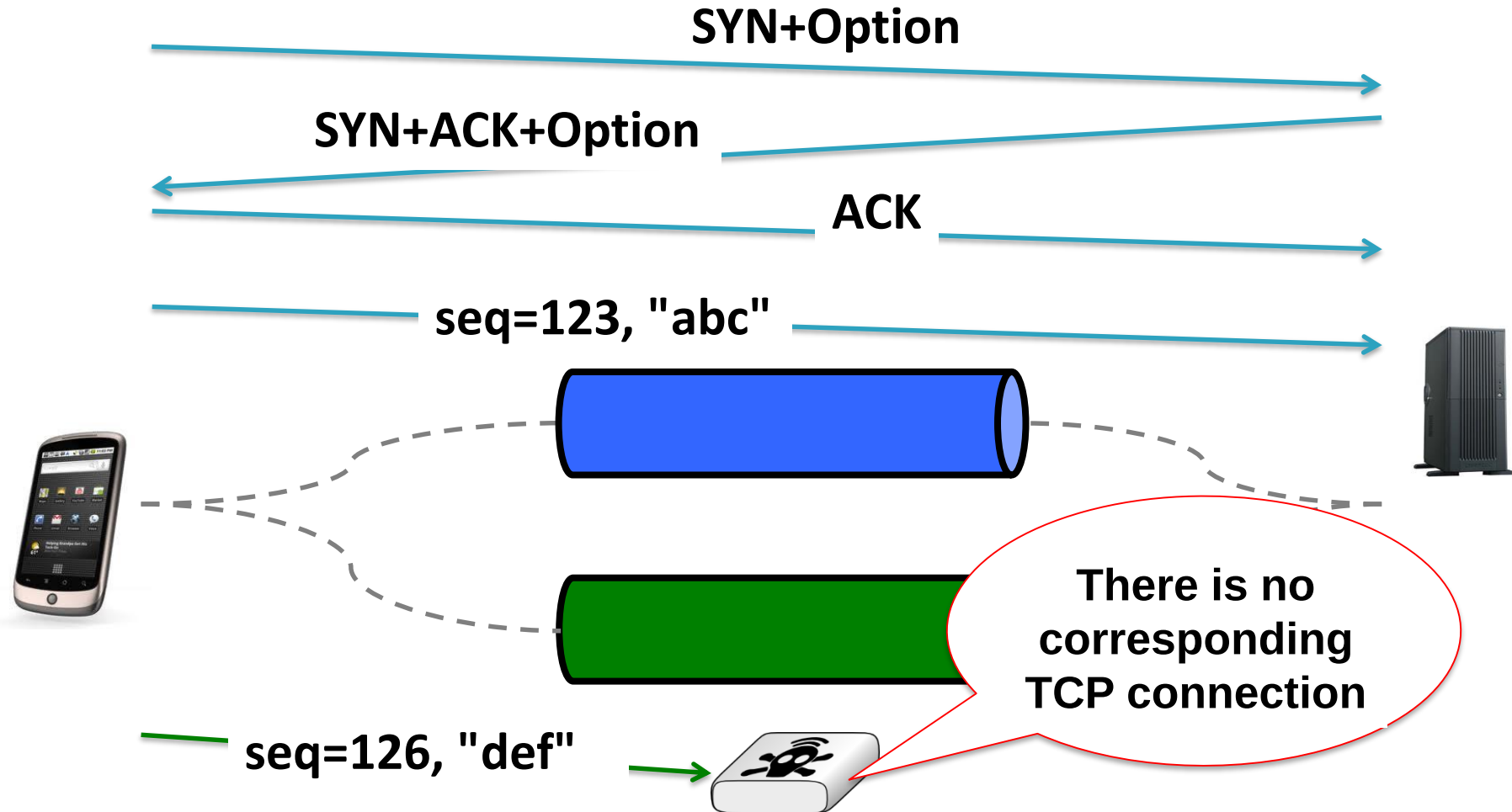
The Multipath TCP protocol

- Control plane
 - How to manage a Multipath TCP connection that uses several paths ?
- Data plane
 - How to transport data ?
- Congestion control
 - How to control congestion over multiple paths ?

A naïve Multipath TCP



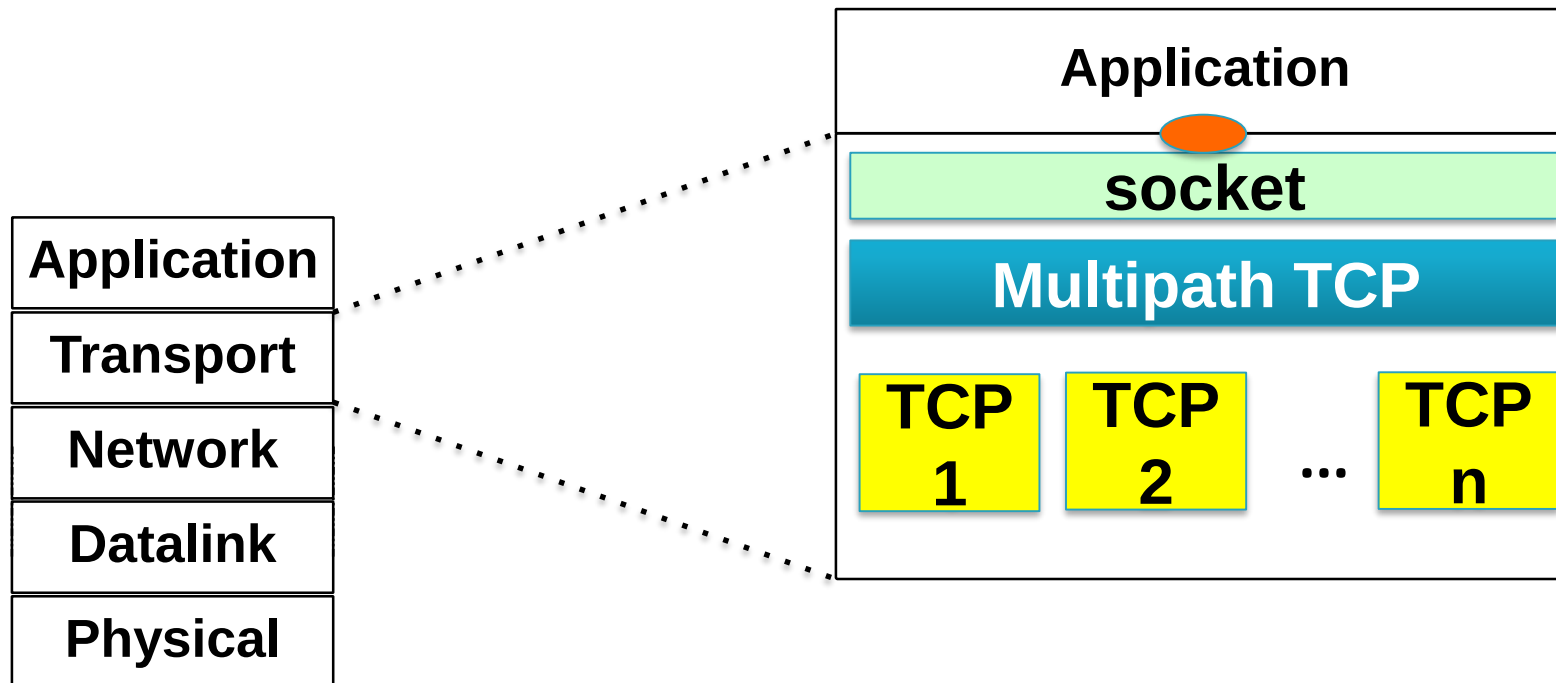
A naïve Multipath TCP In today's Internet ?



Design decision

- *A Multipath TCP connection is composed of one of more regular TCP subflows that are combined*
 - Each host maintains state that glues the TCP subflows that compose a Multipath TCP connection together
 - Each TCP subflow is sent over a single path and appears like a **regular TCP** connection along this path

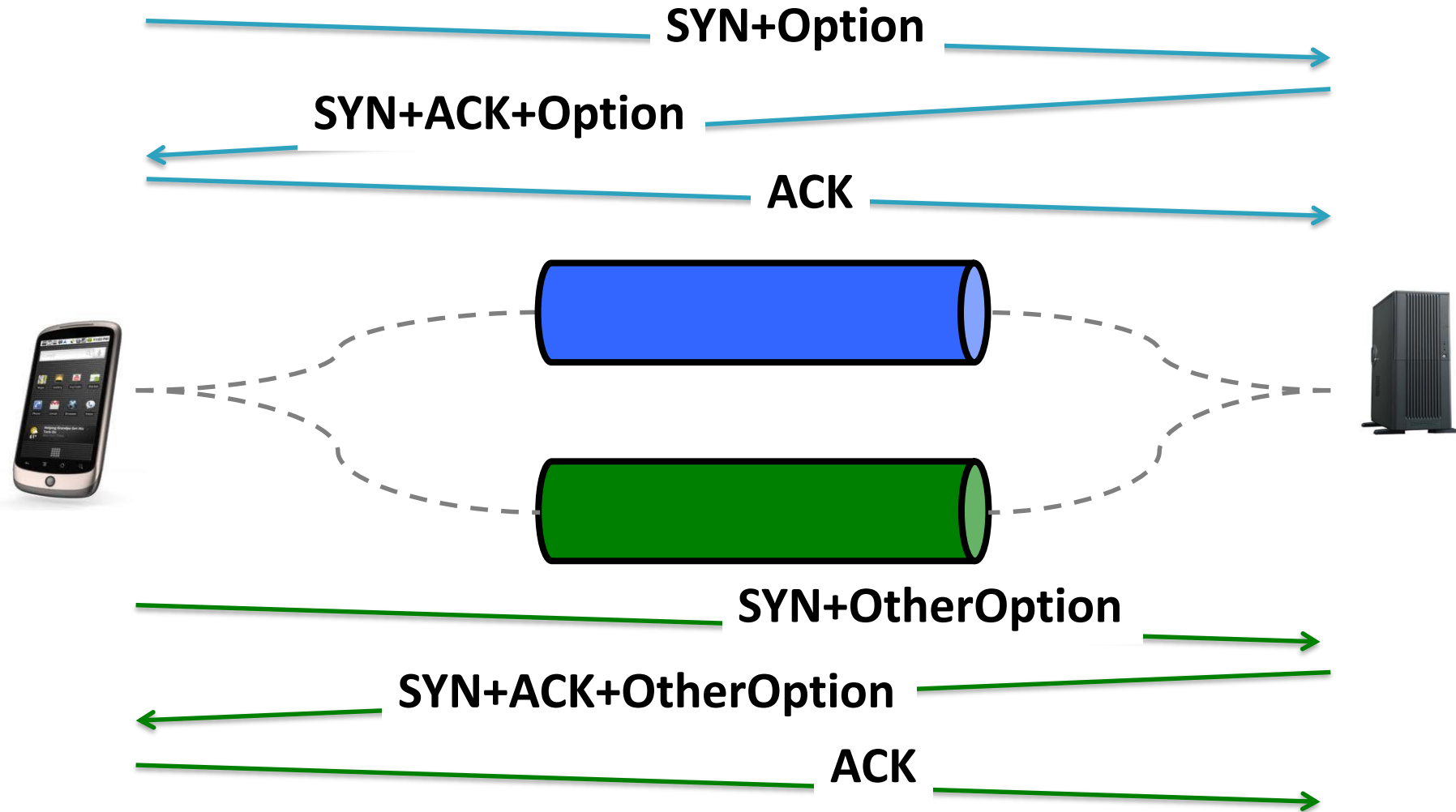
Multipath TCP and the architecture



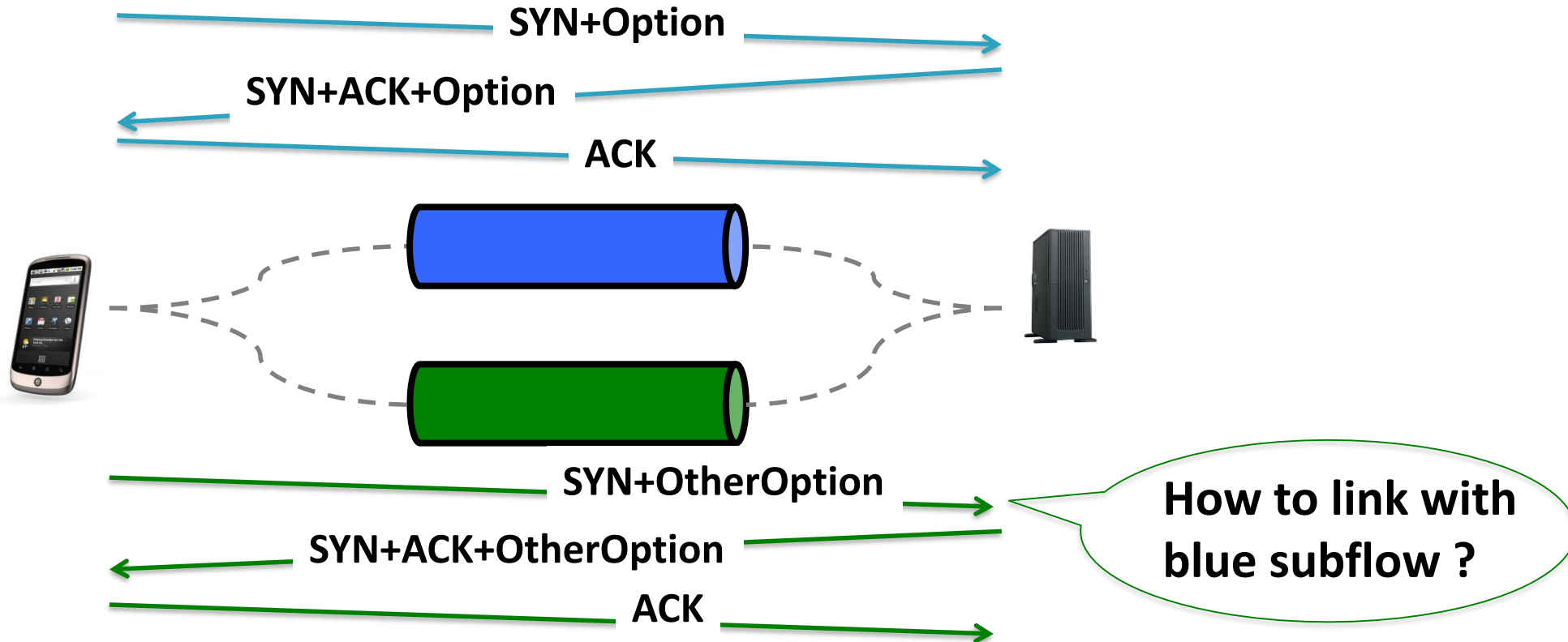
A regular TCP connection

- What is a *regular* TCP connection ?
 - It starts with a three-way handshake
 - SYN segments may contain special options
 - All data segments are sent in sequence
 - There is no gap in the sequence numbers
 - It is terminated by using FIN or RST

Multipath TCP

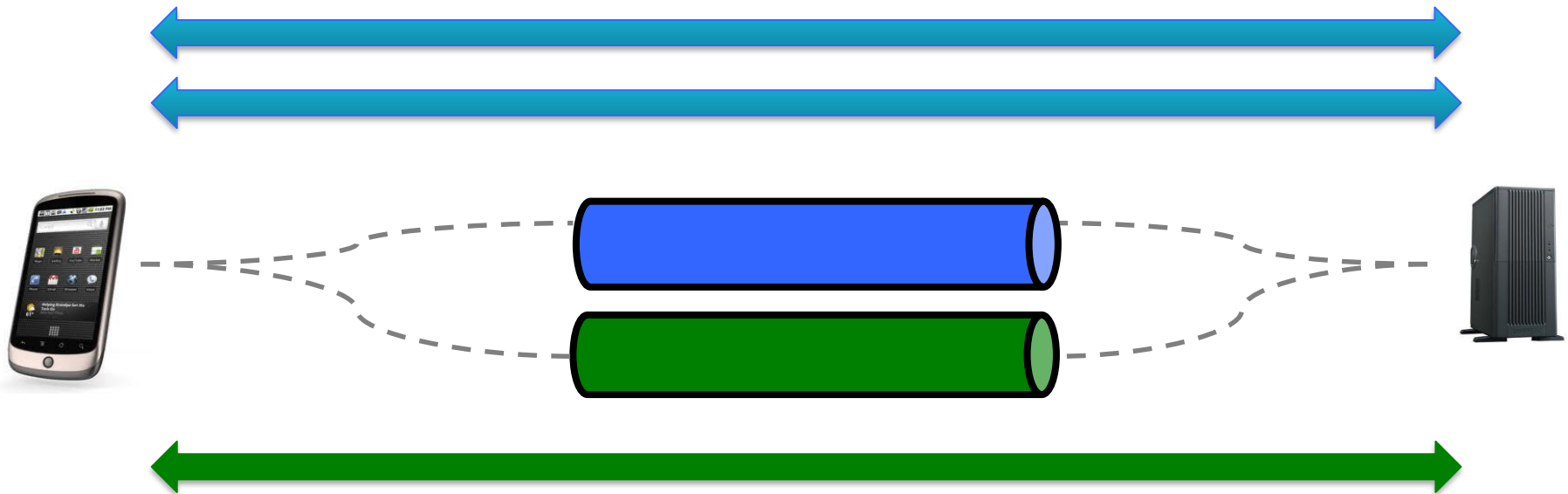


How to combine two TCP subflows ?



Subflow agility

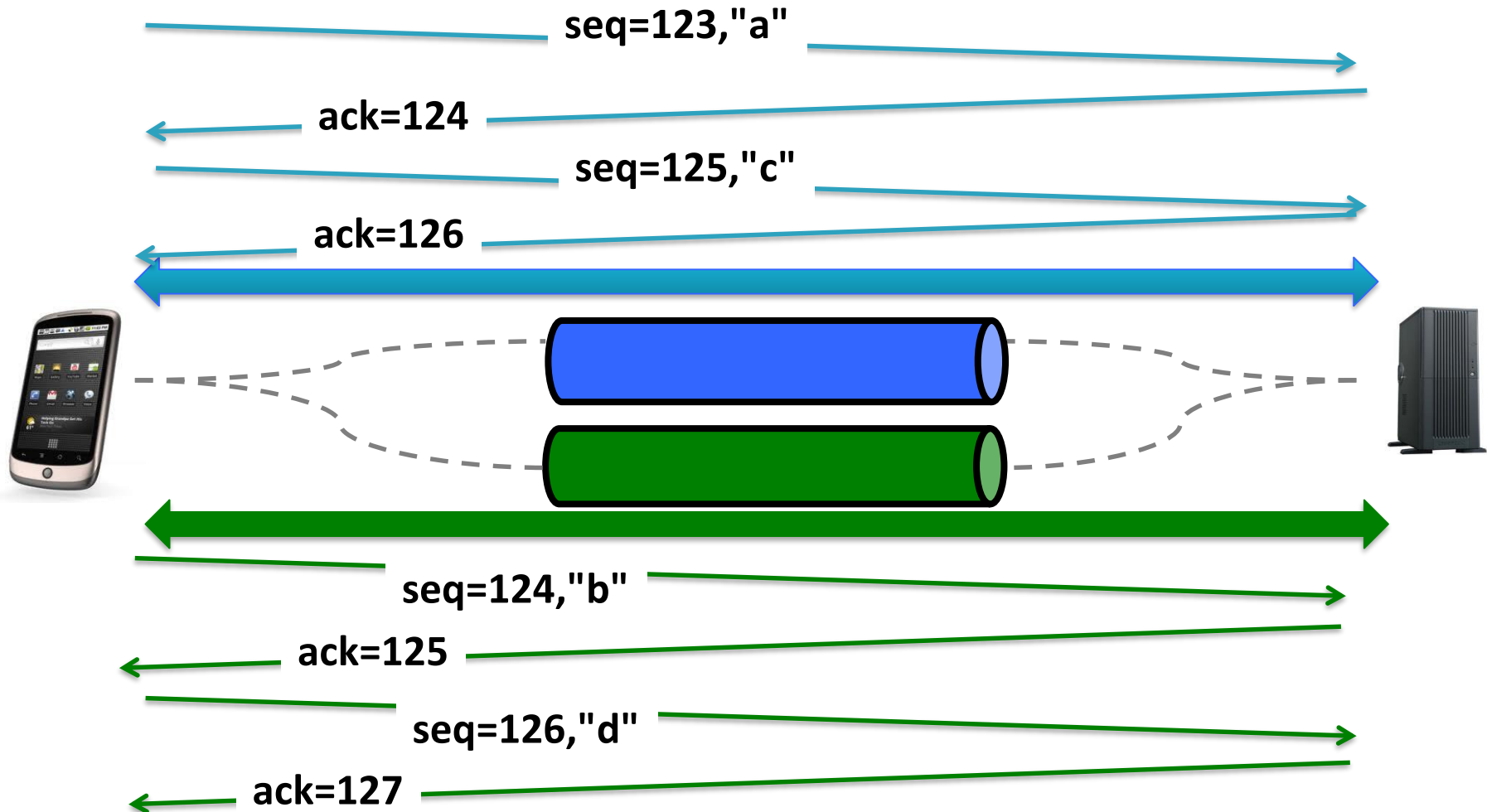
- Multipath TCP supports
 - addition of subflows
 - removal of subflows



The Multipath TCP protocol

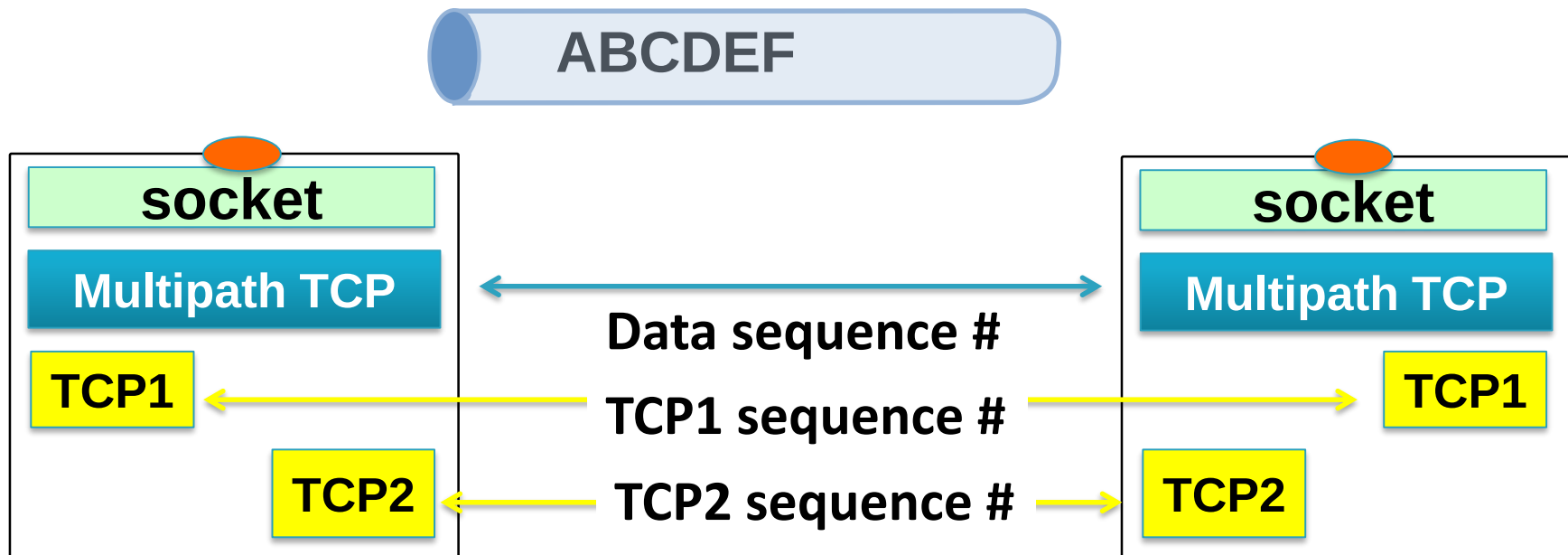
- Control plane
 - How to manage a Multipath TCP connection that uses several paths ?
- Data plane
 - How to transport data ?
- Congestion control
 - How to control congestion over multiple paths ?

How to transfer data ?



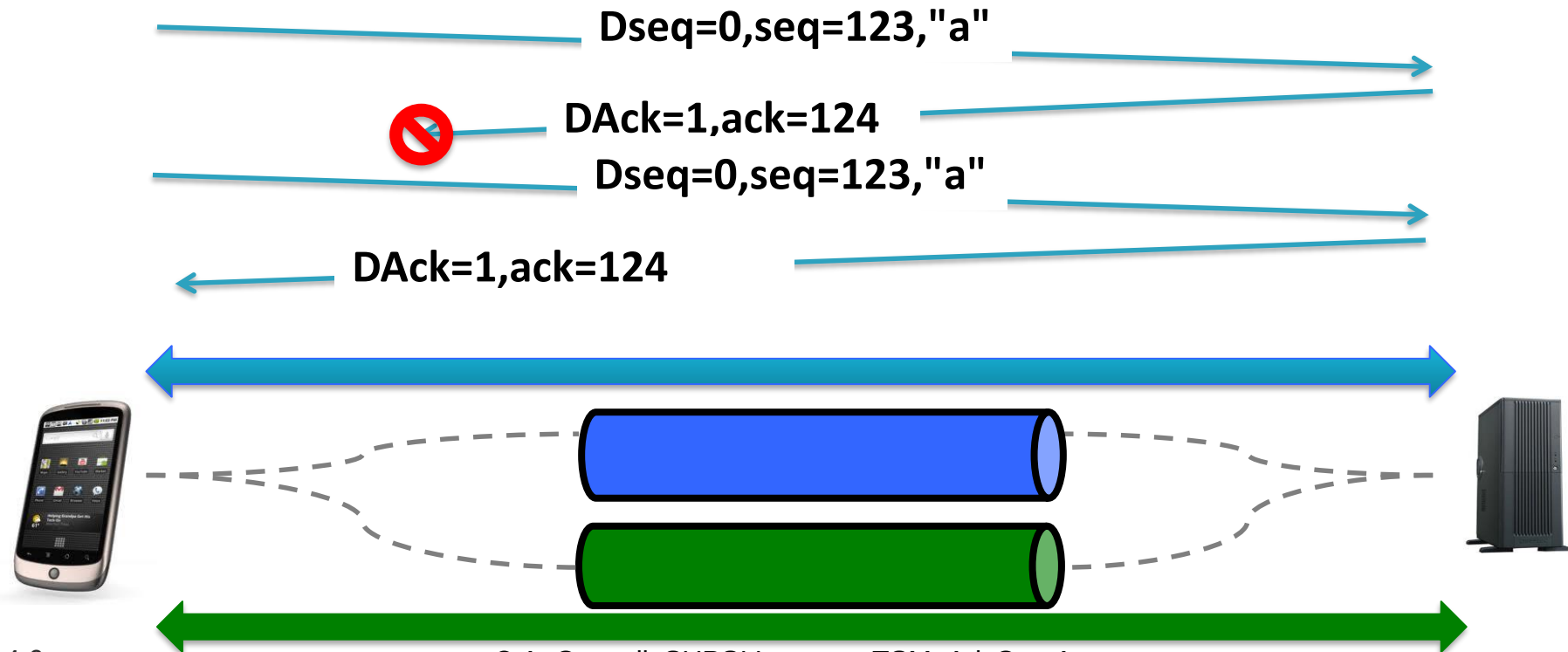
Multipath TCP Data transfer

- Two levels of sequence numbers



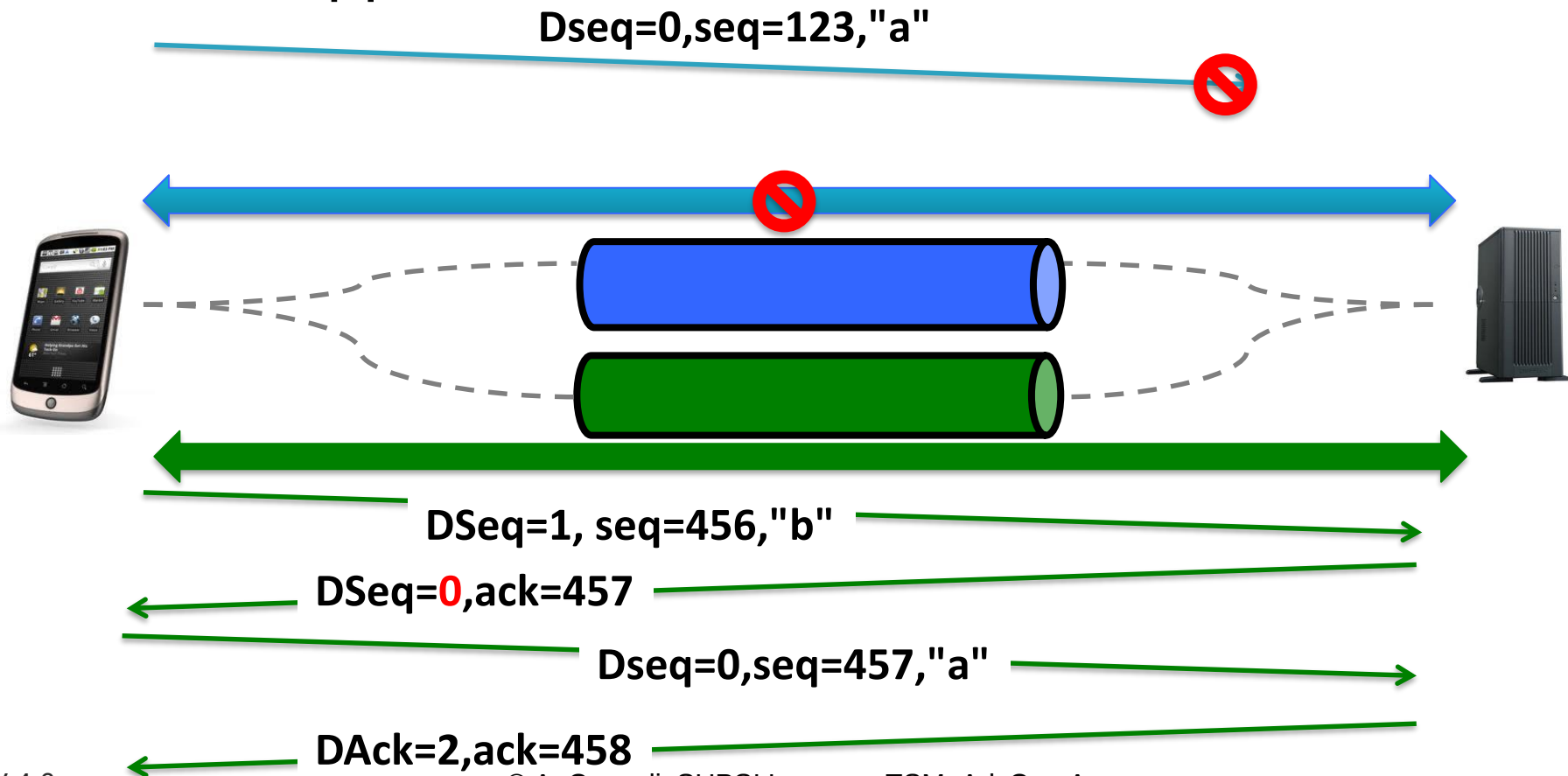
Multipath TCP- How to deal with losses ?

- Data losses over one TCP subflow
 - Fast retransmit and timeout as in regular TCP

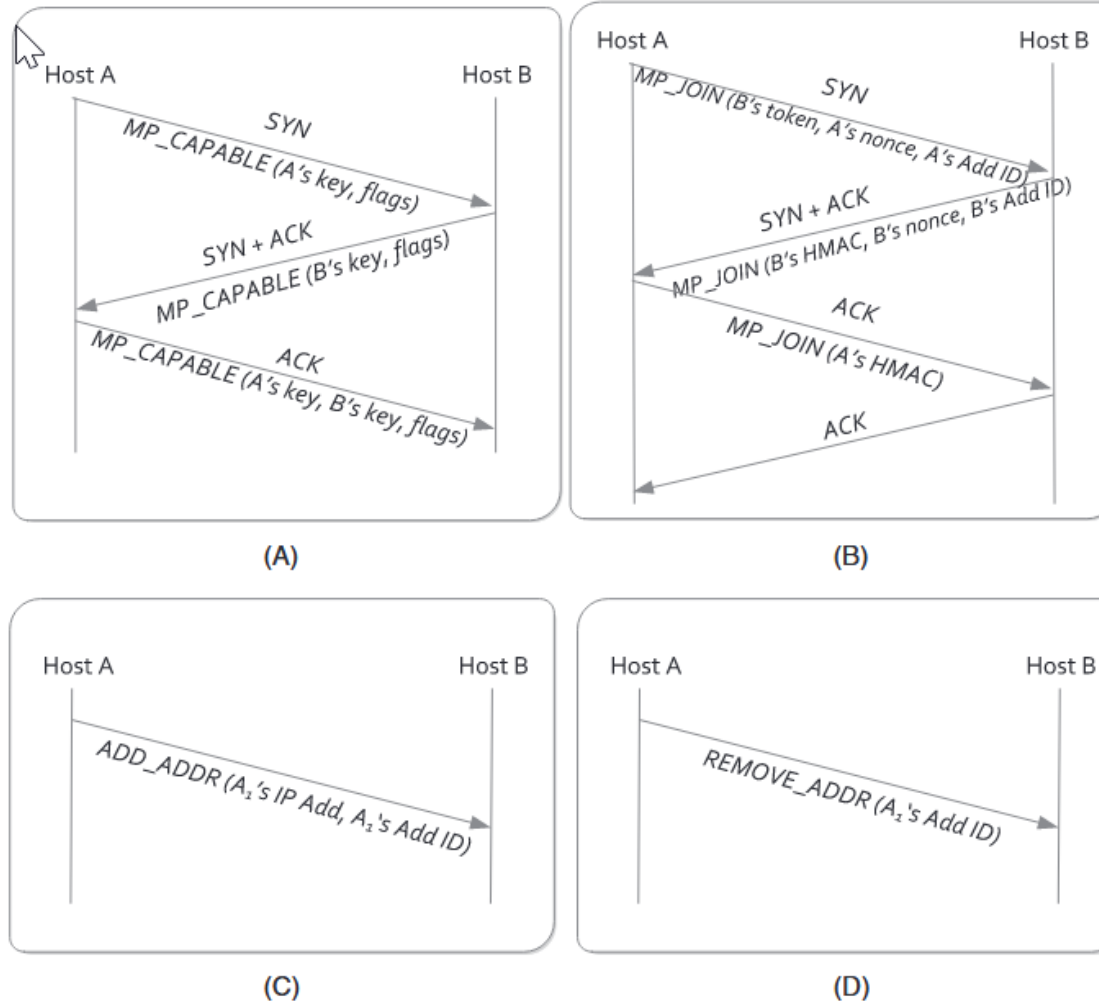


Multipath TCP

- What happens when a TCP subflow fails ?



MPTCP connection(s)



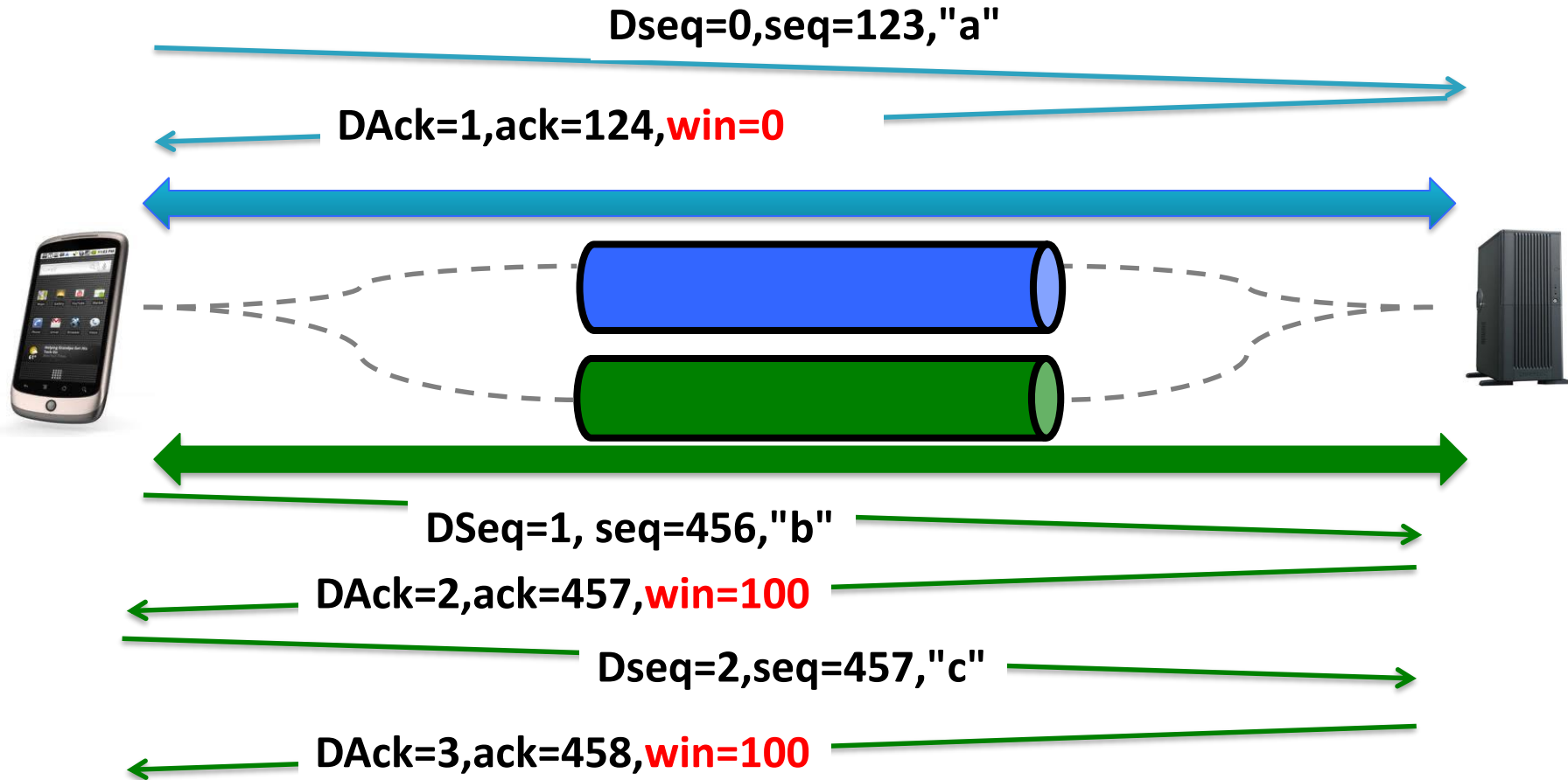
Retransmission heuristics

- Heuristics used by current Linux implementation
 - Fast retransmit is performed on the same subflow as the original transmission
 - Upon timeout expiration, reevaluate whether the segment could be retransmitted over another subflow
 - Upon loss of a subflow, all the unacknowledged data are retransmitted on other subflows

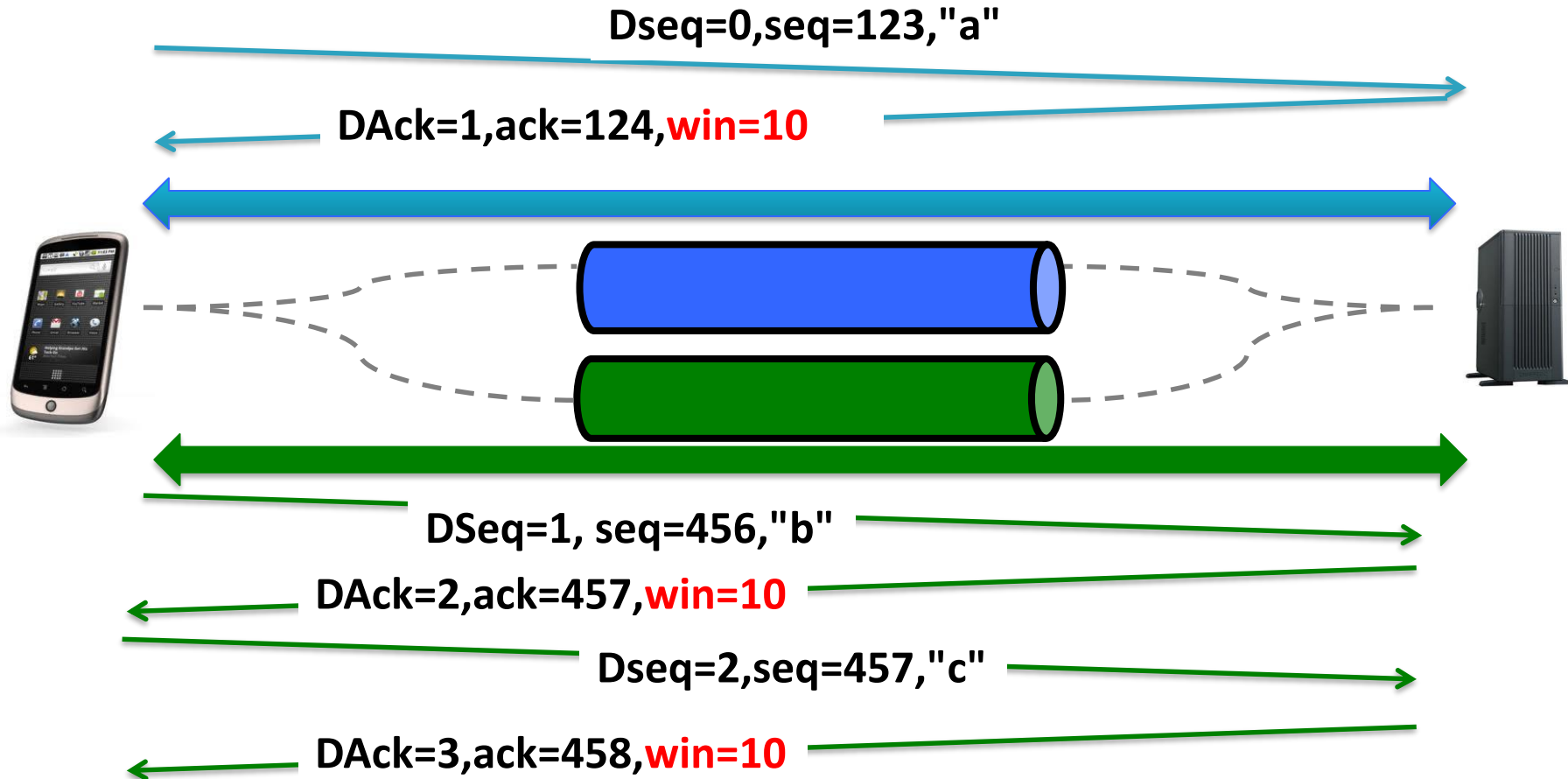
Flow control

- How should the window-based flow control be performed ?
 - Independent windows on each TCP subflow
 - A single window that is shared among all TCP subflows

Independent windows



A single window shared by all subflows



Multipath TCP Windows

- Multipath TCP maintains one window per Multipath TCP connection
 - Window is relative to the last acknowledged data (Data Ack)
 - Window is shared among all subflows
 - It's up to the implementation to decide how the window is shared
 - Window is transmitted inside the `window` field of the regular TCP header
 - If middleboxes change `window` field,
 - use largest `window` received at MPTCP-level
 - use received `window` over each subflow to cope with the flow control imposed by the middlebox

Multipath TCP - Data sequence numbers

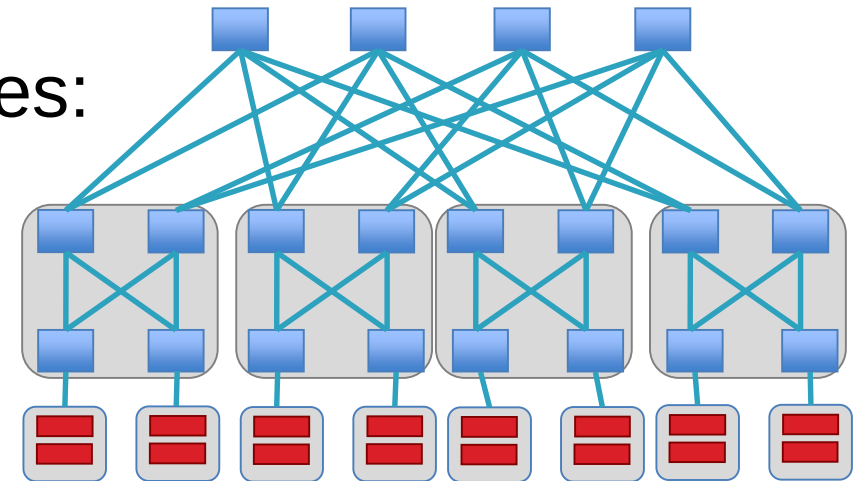
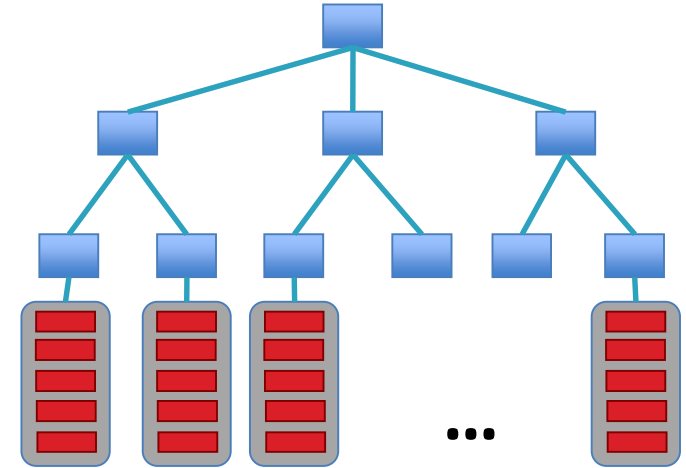
- Data sequence numbers and Data acknowledgements
 - Maintained as 64 bits field
 - Implementations can, as an optimisation, only transmit the lower 32 bits of the data sequence and acknowledgements

Roles of the initial TCP handshake

- Check willingness to open TCP connection
 - Propose initial sequence number
 - Negotiate Maximum Segment Size
- TCP options
 - negotiate Timestamps, SACK, Window scale
- Multipath TCP
 - check that server supports Multipath TCP
 - propose Token in each direction
 - propose initial Data sequence number in each direction
 - Exchange keys to authenticate subflows

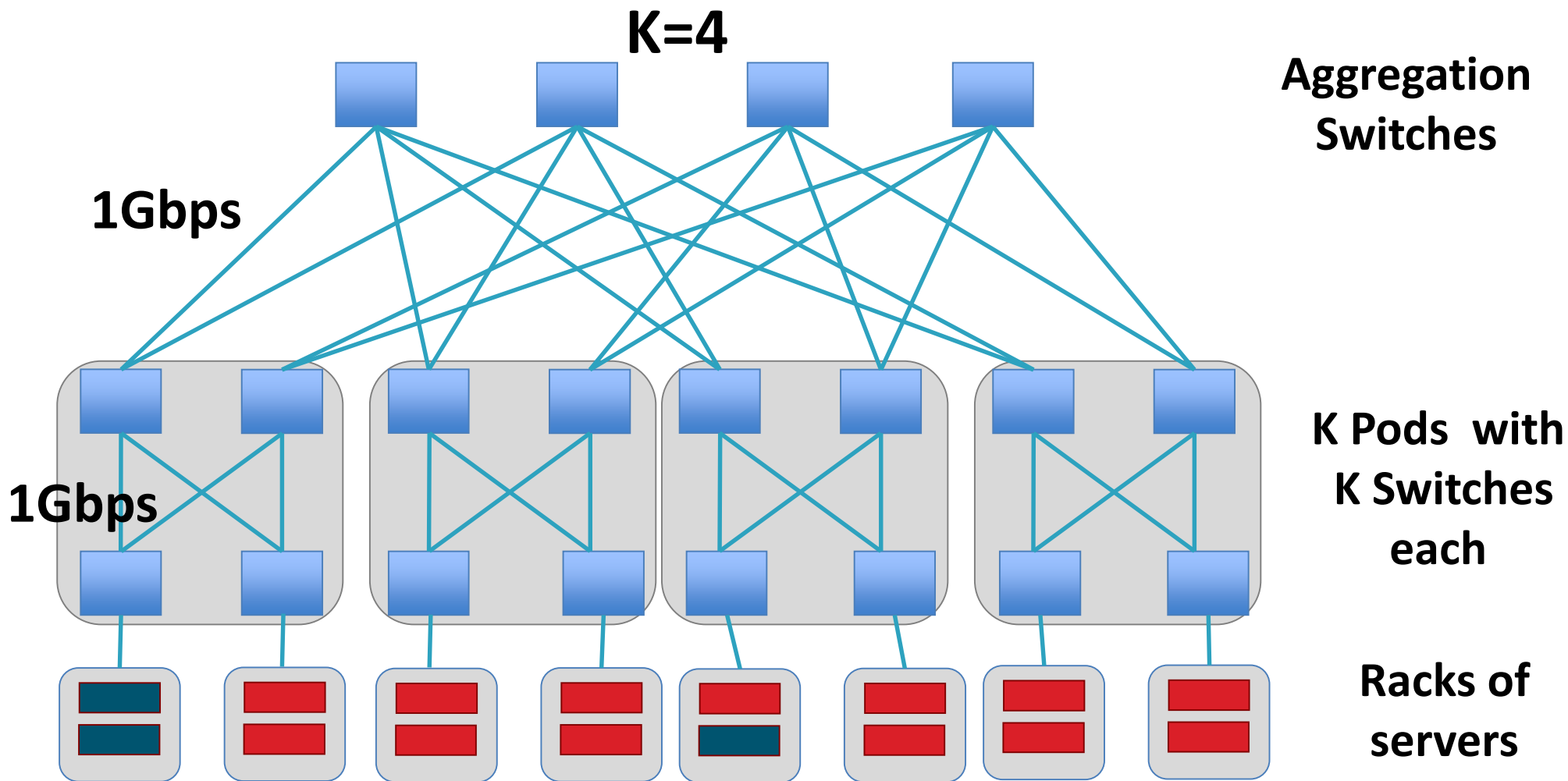
Datacenters evolve

- Traditional Topologies are tree-based
 - Poor performance
 - Not fault tolerant
- Shift towards multipath topologies: FatTree, BCube, VL2, Cisco, EC2

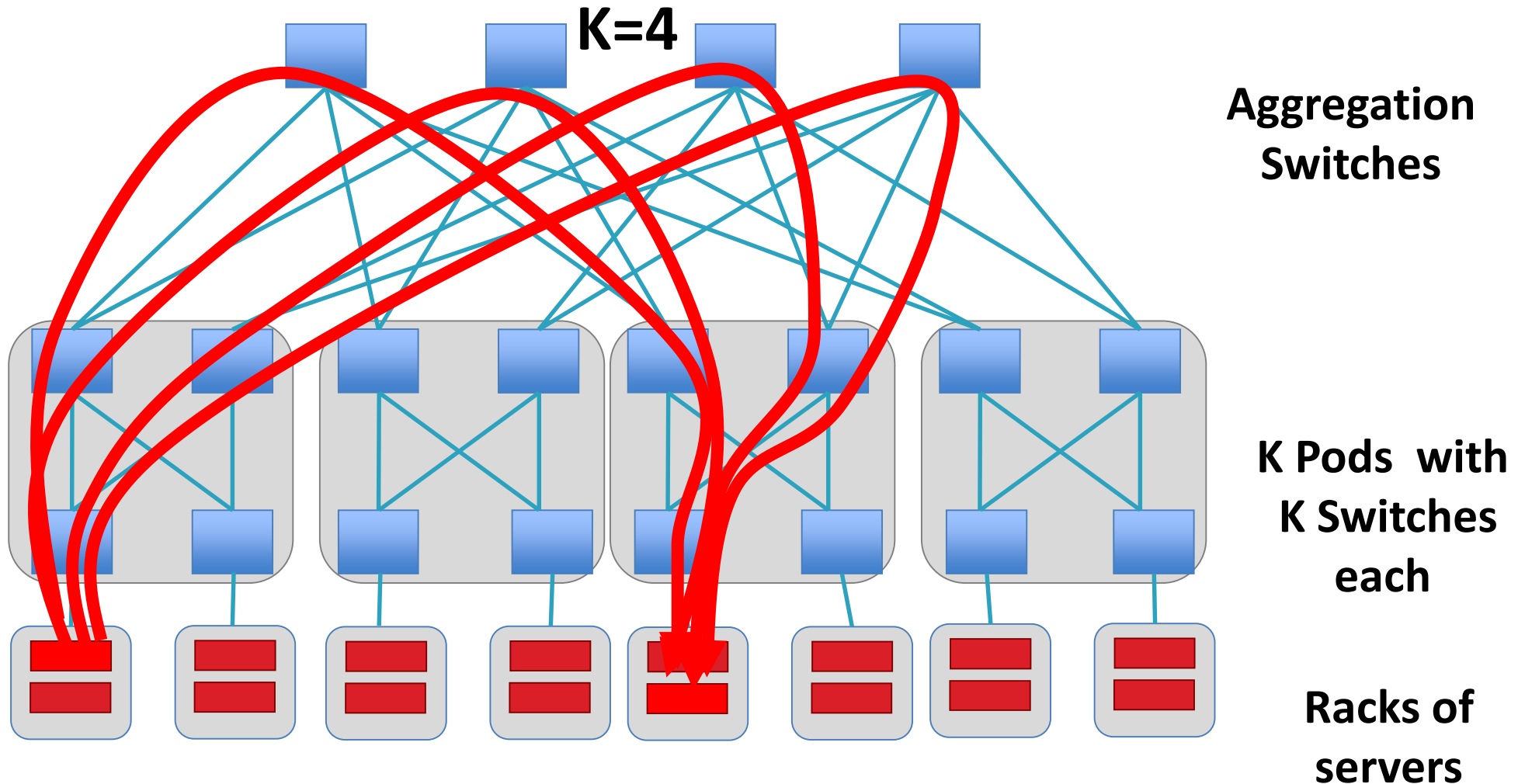


C. Raiciu, et al. "Improving datacenter performance and robustness with multipath TCP," ACM SIGCOMM 2011.

Fat Tree Topology

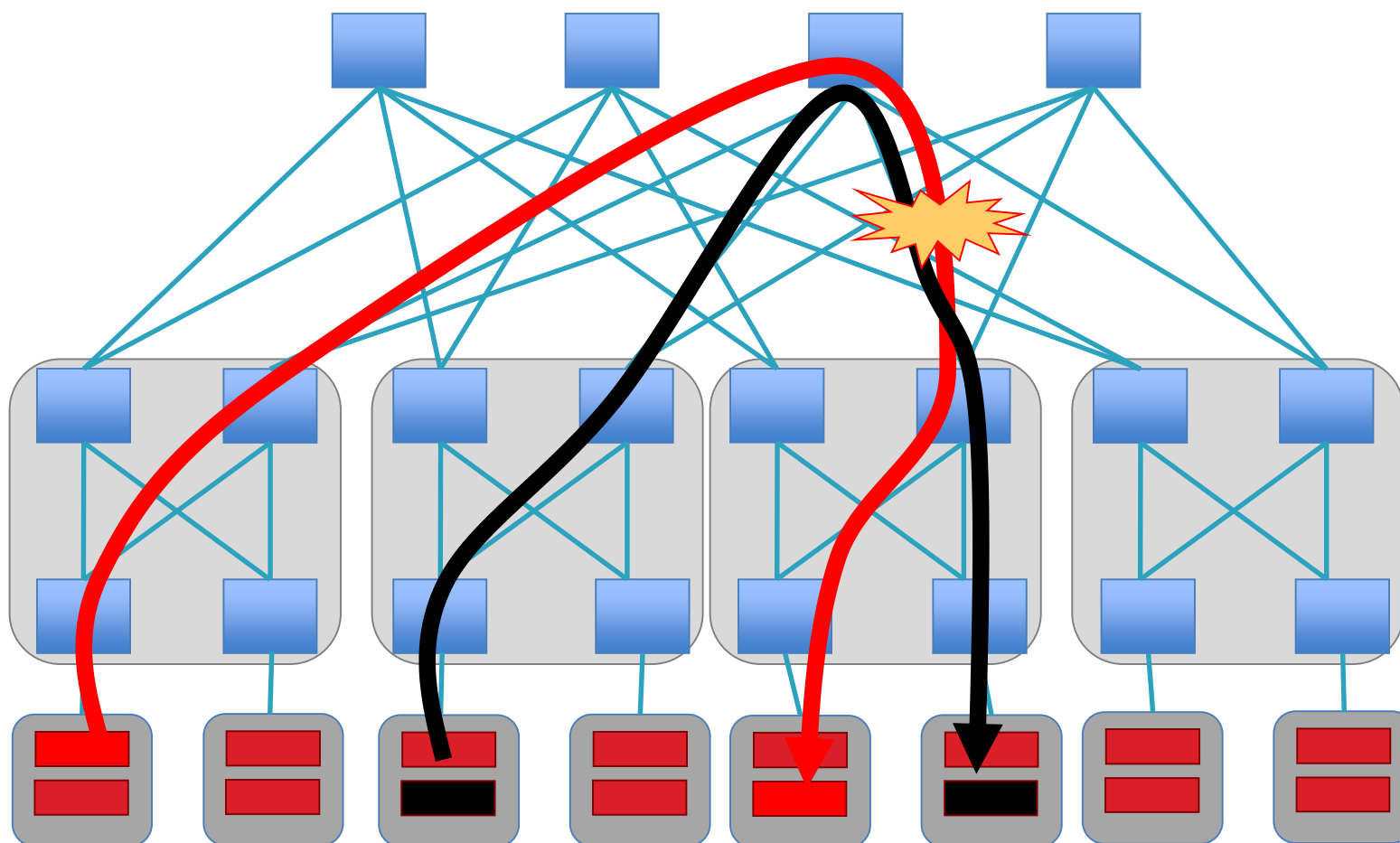


Fat Tree Topology



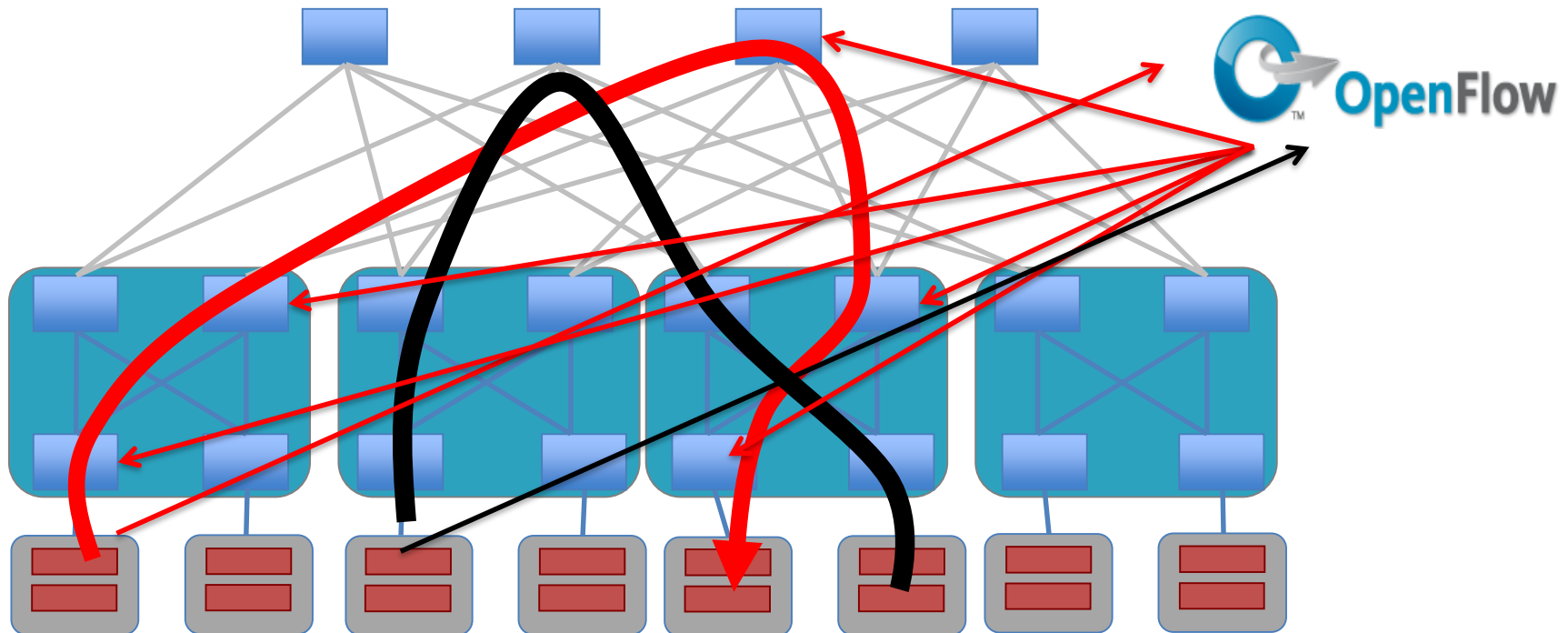
C. Raiciu, et al. "Improving datacenter performance and robustness with multipath TCP," *ACM SIGCOMM* 2011.

Collisions



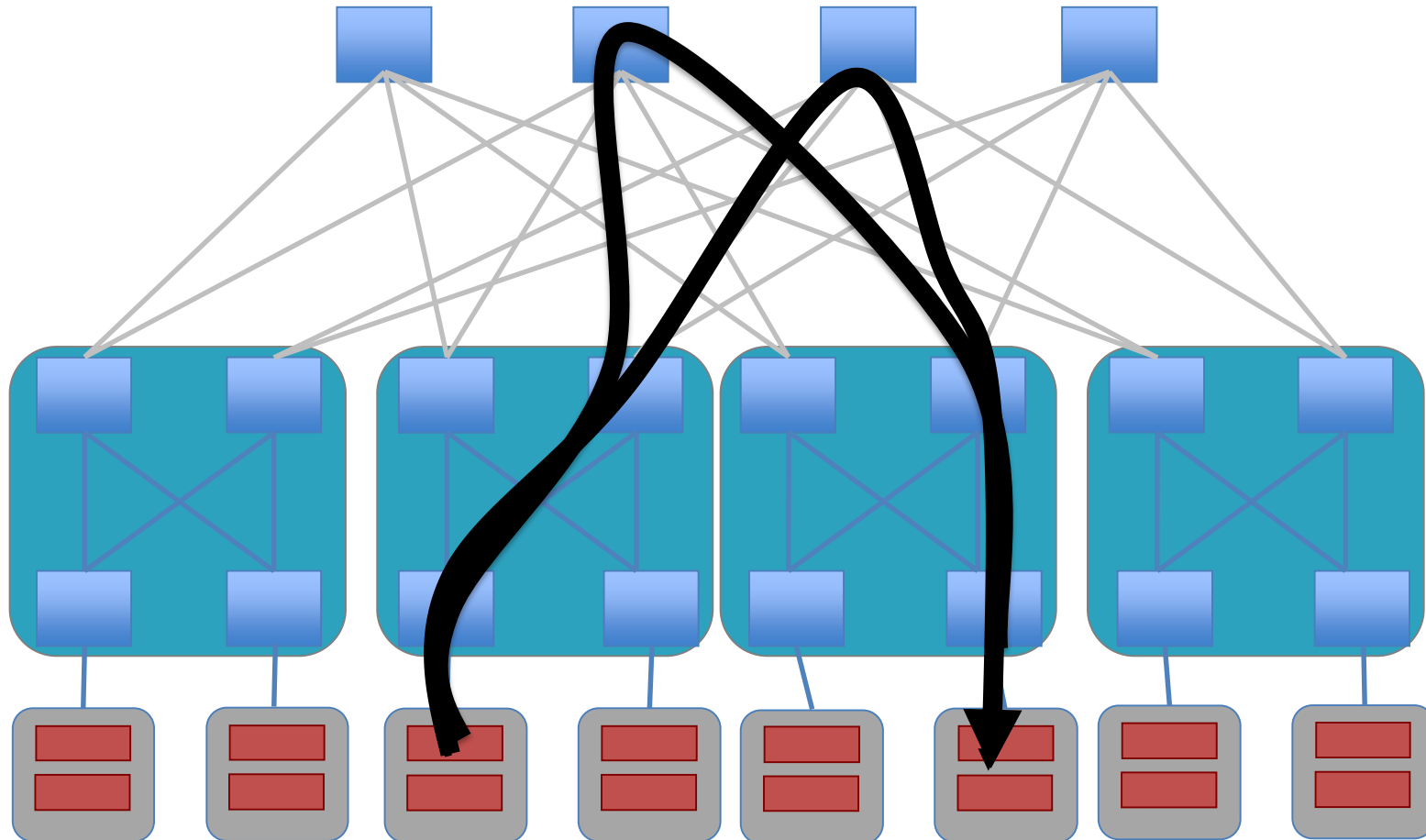
How to get rid of these collisions ?

- Consider TCP performance as an optimisation problem



Can we improve Multipath TCP ?

- Two subflows may follow similar paths



Interesting articles

- Use of Fat Tree in datacenters and supercomputing:
 - Examples: [Sierra](#) , [Summit](#)
- Infiniband: <https://www.infinibandta.org/>
- Fibre Channel:
https://en.wikipedia.org/wiki/Fibre_Channel
- <https://www.techtarget.com/searchstorage/tip/Storage-networking-technologies-explained>

TCP Extensions for Multipath Operation

- Standardized in RFC 8684, MPTCP is now a cornerstone for mission-critical applications. By shifting traffic seamlessly from failing paths to healthy ones, it ensures uninterrupted service for high-stakes environments like **mobile voice processing** and **data center synchronization**
- <https://datatracker.ietf.org/doc/html/rfc8684>

Motivation for MPTCP: mobility + mobile devices

- One device, many IP-enabled interfaces

